

Конспект: что происходит при входе в систему

Пошаговый разбор процесса авторизации в проекте multi-tenant-ecommerce: от нажатия клавиш в поле email/пароль на фронтенде (platform-admin, React + Redux Toolkit) до записи сессии в базу данных на бэкенде (NestJS + Prisma + PostgreSQL). Основан на файле `AUTH_FLOW.md` в корне проекта.

Содержание

1. Архитектура целиком
2. Ввод email на фронтенде
3. Ввод пароля на фронтенде
4. Отправка формы (submit)
5. Что происходит на бэкенде до контроллера
6. `AuthController.login`
7. `AuthService.login` — логика проверки логина и пароля
8. Выдача токенов и создание сессии
9. Ответ приходит на фронтенд
10. Защищённый маршрут и запрос `/auth/me`
11. Что происходит при неверном email / пароле
12. Механизм тихого обновления токена (refresh)
13. Итоговые свойства безопасности

1. Архитектура целиком

- 1 Пользователь печатает email и пароль в контролируемых полях формы (`LoginPage.tsx`).
- 2 Клик по «Sign in» → `handleSubmit` → Redux-thunk `fetchLogin({ email, password })`.
- 3 `authApi.login()` отправляет `POST /auth/login` через общий экземпляр Axios.
- 4 Nest.js: helmet → глобальный `ValidationPipe` (проверка `LoginDto`) → проверка CORS.
- 5 `AuthController.login` передаёт email/пароль и контекст (IP, User-Agent) в `AuthService.login`.
- 6 `AuthService` ищет пользователя по email в Prisma, сравнивает пароль через `bcrypt.compare`, проверяет статус аккаунта.
- 7 При успехе — создаётся сессия в таблице `user_sessions`, подписываются access- и refresh-JWT.
- 8 Ответ `{ accessToken, refreshToken, userType }` возвращается на фронтенд.
- 9 Фронтенд сохраняет токены в `localStorage`, обновляет Redux-состояние, делает `navigate("/dashboard")`.
- 10 `ProtectedRoute` сразу запрашивает `GET /auth/me` с заголовком `Authorization: Bearer <accessToken>`, чтобы получить профиль пользователя.

2. Ввод email на фронтенде

Файл: `platform-admin/src/pages/auth/LoginPage.tsx`

- Поле email — **контролируемый компонент**: его значение хранится в состоянии React `const [email, setEmail] = useState("")`, и каждое нажатие клавиши вызывает `onChange={(e) => setEmail(e.target.value)}`.
- Пока пользователь печатает, **никакие запросы на сервер не отправляются** — нет живой проверки «существует ли такой email», нет debounce, нет асинхронной валидации.
- У поля есть атрибуты `type="email"`, `autoComplete="username"` и `required`. Это нативные HTML-атрибуты: браузер сам откажется отправлять форму (и покажет собственную подсказку), если поле пустое или не похоже на email — **ещё**

до вызова `handleSubmit`. Это чисто UX-помощь на клиенте, а не механизм безопасности — бэкенд всё равно валидирует независимо (раздел 5).

- Пока идёт запрос (`isSubmitting === true`), поле блокируется атрибутом `disabled`, чтобы пользователь не мог редактировать его во время отправки.

3. Ввод пароля на фронтенде

Тот же паттерн, что и с email:

- Контролируемое поле, привязанное к состоянию `password`, обновляется через `onChange`.
- `type="password"` визуально маскирует символы; `autocomplete="current-password"` подсказывает браузерному менеджеру паролей, что это поле для входа.
- `required` запускает нативную валидацию при попытке отправки пустого поля.
- На форме входа **нет** клиентской проверки сложности пароля (в отличие от формы регистрации) — DTO на бэкенде требует лишь непустую строку (раздел 5). Реальная корректность пароля становится известна только после сравнения с хешем на сервере.

4. Отправка формы (submit)

`handleSubmit` в `LoginPage.tsx` делает следующее:

- `e.preventDefault()` — останавливает нативную перезагрузку/навигацию формы браузером.
- `setIsSubmitting(true)` и `setError(null)` — сбрасывают состояние UI, блокируют поля и кнопку, меняют текст кнопки на спиннер «Signing in...».
- `await dispatch(fetchLogin({ email, password })).unwrap()` — диспатчит Redux Toolkit **async thunk** (`store/auth/authSlice.ts`). Вызов `.unwrap()` делает так, что промис либо резолвится с реальными данными, либо реджектится со значением из `rejectWithValue` — это позволяет ловить ошибку обычным `try/catch`.
- При успехе: `navigate("/dashboard")`.
- При ошибке: пойманное значение (строка) кладётся в состояние `error` и выводится под полями формы.
- `finally` всегда возвращает `isSubmitting` обратно в `false`.

4.1 Что делает `thunk fetchLogin`

```
authApi.login(payload) -> POST /auth/login
```

- `authApi.login` (`platform-admin/src/api/auth.ts`) вызывает общий экземпляр `Axios` (`lib/axios.ts`) методом `POST /auth/login` с телом `{ email, password }` и возвращает `response.data`.
- Request-интерцептор:** перед отправкой `Axios` проверяет `isPublicAuthPath(config.url)`. Пути `/auth/login` и `/auth/refresh` находятся в списке публичных, поэтому **заголовок `Authorization` не добавляется** к этому запросу (токена ещё и нет).
- Запрос уходит на адрес из `VITE_APP_API_BASE_URL` (по умолчанию `http://localhost:4000`) — обычный кросс-доменный HTTP-вызов от dev-сервера `platform-admin` к Nest API.

5. Что происходит на бэкенде до контроллера

Файл: `backend/src/main.ts`

- `helmet()` — проставляет усиленные security-заголовки на каждый ответ (CSP, X-Frame-Options и т.д.).
- Глобальный `ValidationPipe`:**
 - `whitelist: true` — удаляет из тела запроса любые поля, не описанные в DTO.
 - `forbidNonWhitelisted: true` — если есть лишнее поле, запрос отклоняется с `400 Bad Request` вместо тихого удаления.

- `transform: true` — сырое JSON-тело превращается в реальный экземпляр класса `LoginDto`, и срабатывают декораторы `class-validator`: `email` должен пройти `@IsEmail()`, `password` — `@IsString()` и `@IsNotEmpty()`.
 - Если валидация не прошла, пайп выбрасывает исключение ещё до вызова `AuthController.login`, и ответ — `400` с массивом сообщений (например, `["email must be an email"]`).
3. **CORS** включается только если задана переменная окружения `CORS_ORIGINS`; dev-адрес `platform-admin` должен входить в этот список через запятую, а `credentials: true` разрешает передачу заголовка `Authorization` / `cookies` между доменами.
4. **AllExceptionsFilter** — «последний рубеж»: любое исключение из любого места запроса (включая `AuthService`) приводится к единому виду `{ statusCode, message, error, path, timestamp }`. Для «настоящей» серверной ошибки (не `HttpException`) реальное сообщение намеренно заменяется на «Internal server error», чтобы стек вызовов/фрагменты SQL никогда не утекали клиенту; при этом настоящая ошибка логируется на сервере.

6. AuthController.login

Файл: `backend/src/auth/auth.controller.ts`

```
@Post('login')
@HttpCode(HttpStatus.OK)
login(@Body() loginDto: LoginDto, @Ip() ip: string,
      @Headers('user-agent') userAgent?: string)
```

- Маршрут публичный (нет `@UseGuards(JwtAuthGuard)`).
- Явно возвращает `200 OK` вместо стандартного для POST у Nest `201 Created`.
- Забирает IP звонящего (`@Ip()`) и заголовок `User-Agent` — они передаются дальше как «контекст входа» только для записи в сессию (метаданные устройства/IP), а не для решений об аутентификации.
- Всю логику делегирует `AuthService.login(email, password, context)`.

7. AuthService.login — логика проверки логина и пароля

Файл: `backend/src/auth/auth.service.ts`

```
const user = await this.userService.findByEmail(email);
const passwordMatches = await bcrypt.compare(
  password,
  user?.passwordHash ?? ABSENT_USER_PASSWORD_HASH,
);
if (!user || !passwordMatches) {
  throw new BadRequestException('Invalid email or password');
}
if (user.status !== UserStatus.ACTIVE) {
  throw new ForbiddenException('This account is not active');
}
```

По шагам:

1. **findByEmail** (`users/users.service.ts`) выполняет `prisma.user.findUnique({ where: { email } })`. Колонка `email` имеет тип `Postgres Citext` (регистронезависимый текст), поэтому `Foo@Example.com` совпадёт с сохранённым `foo@example.com` — один SQL `SELECT` к таблице `users`.
2. **Проверка пароля**: `bcrypt.compare` выполняется всегда, даже если `user === null`. В этом случае сравнение идёт с зашитым в код фиктивным bcrypt-хешем (`ABSENT_USER_PASSWORD_HASH`). Это намеренная **защита от timing-атак / enumeration пользователей**: без неё запрос с несуществующим email вернулся бы мгновенно (сравнивать не с чем), а запрос с реальным email занимал бы ~100 мс, которые требует bcrypt — по разнице во времени ответа

злоумышленник мог бы понять, какие email существуют в системе. Сравнение с фиктивным хешем выравнивает время ответа в обоих случаях.

3. Что бы ни оказалось неверным — email не найден или пароль не совпал — ответ **одинаковый**: сообщение `"Invalid email or password"` и статус `400 Bad Request`. Фронтенд (и потенциальный атакующий) не может понять, что именно было неверным.
4. Если пароль и email верны, но статус аккаунта не `ACTIVE` (например, `SUSPENDED`), выбрасывается `403 Forbidden` («This account is not active») — это первая точка, где сообщение об ошибке действительно отличается, но только после того, как пароль уже подтверждён как верный.
5. При успехе вызывается `issueTokenPair(user, context)`.

8. Выдача токенов и создание сессии

```
async issueTokenPair(user, context) {
  const sessionId = createId(); // cuid2
  const tokens = await this.signTokenPair(user, sessionId);
  await this.prisma.userSession.create({
    data: {
      id: sessionId,
      userId: user.id,
      deviceLabel: context.deviceLabel, // User-Agent
      ipAddress: context.ipAddress,
      refreshTokenHash: this.hashRefreshToken(tokens.refreshToken),
      expiresAt: this.refreshTokenExpiresAt(tokens.refreshToken),
    },
  });
  return tokens;
}
```

- Генерируется новый непрозрачный **id сессии** (`cuid2`) — это не сам JWT, а идентификатор строки в таблице `user_sessions`, благодаря которому сессию можно отозвать (logout) независимо от срока жизни JWT.
- **signTokenPair** параллельно подписывает два JWT через `JwtService`:
 - **Access-токен**: payload `{ userId, email, userType, sid: sessionId }`, подписан секретом `JWT_ACCESS_SECRET`, срок жизни — `JWT_ACCESS_EXPIRES_IN`.
 - **Refresh-токен**: payload `{ userId, sessionId }`, подписан другим секретом `JWT_REFRESH_SECRET`, срок жизни — `JWT_REFRESH_EXPIRES_IN` (обычно намного дольше).
 - Использование двух разных секретов означает, что утёрший access-токен нельзя использовать для подделки refresh-токена, и наоборот.
- **Refresh-токен никогда не хранится в открытом виде**. Функция `hashRefreshToken` вычисляет `HMAC-SHA256(JWT_REFRESH_SECRET, "refresh-token:" + refreshToken)`, и в колонку `user_sessions.refresh_token_hash` записывается только этот дайджест. Даже при полной утечке базы данных злоумышленник не получит пригодные для использования refresh-токены.
- `expiresAt` вычисляется из claim'a `exp` самого refresh-JWT, поэтому срок действия записи в БД всегда совпадает с реальным сроком жизни токена.
- В таблицу `user_sessions` добавляется одна строка (`ip_address`, `device_label` берутся прямо из IP/User-Agent запроса, захваченных в контроллере).
- Метод возвращает `{ accessToken, refreshToken }`; `login()` добавляет `userType` и возвращает `{ accessToken, refreshToken, userType }` как итоговое тело HTTP-ответа.

9. Ответ приходит на фронтенд

В `platform-admin/src/store/auth/authSlice.ts`, thunk `fetchLogin`:

```
const tokens = await authApi.login(payload);
if (!isCompleteLoginResponse(tokens)) {
  return rejectWithValue("Could not Sign in: incomplete response");
}
storage.setItem(ACCESS_TOKEN, tokens.accessToken);
storage.setItem(REFRESH_TOKEN, tokens.refreshToken);
storage.setItem(USER_TYPE, tokens.userType);
return tokens;
```

- `isCompleteLoginResponse` — рантайм type guard: заново проверяет, что `accessToken` / `refreshToken` — непустые строки, а `userType` — известное значение перечисления, на случай если бэкэнд когда-нибудь вернёт неполное/некорректное тело.
- Оба токена и `userType` записываются в `localStorage` под версионированными ключами (`auth:v1:access-token` и т.п.) через `lib/storage.ts`, который оборачивает каждое чтение/запись в `try/catch` (приватный режим браузера / переполнение хранилища деградируют без падения приложения).
- В `fetchLogin.fulfilled` Redux-слайс устанавливает `accessToken`, `refreshToken`, `userType` и `status: "succeeded"`.
- В `LoginPage` вызов `await ... .unwrap()` резолвится, и срабатывает `navigate("/dashboard")`.
- Если что-то из перечисленного выше падает (ошибка сети, 400, 403, некорректный ответ), ветка `catch` вызывает `storage.clearSession()` (удаляет протухшие токены) и реджектится с человекочитаемым сообщением, извлечённым функцией `getAxiosErrorMessage`, которая читает `error.response.data.message` (строку или первый элемент массива ошибок валидации) и в крайнем случае возвращает сообщение по умолчанию. Именно эта строка показывается под формой входа.

10. Защищённый маршрут и запрос `/auth/me`

`platform-admin/src/components/ProtectedRoute.tsx` защищает `/dashboard`:

- Читает `accessToken`, `status`, `user` из Redux.
- Если токен есть, при монтировании диспатчит `fetchMe()`.
- `fetchMe` вызывает `GET /auth/me`. На этот раз путь **не входит** в список публичных, поэтому request-интерцептор Axios прикрепляет `Authorization: Bearer <accessToken>`.
- Пока `user` не заполнен, `ProtectedRoute` показывает заглушку «Loading...»; если токена нет, либо thunk завершился с ошибкой, происходит редирект на `/auth/login`.

10.1 Что происходит на бэкенде при запросе `/auth/me`

- `@UseGuards(JwtAuthGuard)` запускает Passport-стратегию `jwt` (`auth/jwt.strategy.ts`) **до** вызова метода контроллера:
 1. Извлекает bearer-токен из заголовка `Authorization`.
 2. Проверяет его подпись и срок действия по секрету `JWT_ACCESS_SECRET`.
 3. `validate(payload)` загружает строку сессии по `payload.sid`, отклоняя запрос с `401`, если сессии не существует, она отмечена как `revokedAt`, или истёк `expiresAt` (это даёт возможность реализовать logout / принудительный отзыв сессии, даже если сам JWT ещё не истёк).
 4. Отклоняет запрос с `401`, если статус пользователя больше не `ACTIVE`.
 5. Возвращает `{ userId, email, userType, sessionId }`, что Nest/Passport прикрепляет к `request.user`.
- `AuthController.me` читает это через параметр-декоратор `@CurrentUser()` и вызывает `authService.me(userId)`, который делает `prisma.user.findUnique({ where: { id }, omit: { passwordHash: true, twoFactorSecret: true } })` — хеш пароля и секрет 2FA **никогда** не сериализуются ни в один ответ API.
- Фронтенд сохраняет полученный профиль в `state.auth.user`, дополнительно вычисляя на клиенте `avatarName` (инициалы) из `fullName` / `email`.

11. Что происходит при неверном email / пароле

Ситуация	Где перехватывается	HTTP-статус	Сообщение пользователю
Email не проходит формат <code>@IsEmail()</code> (например, нет «@»)	ValidationPipe, до контроллера	400	первое сообщение валидатора, напр. «email must be an email»
Пустой пароль	ValidationPipe	400	«password should not be empty»
Лишнее/неожиданное поле в теле запроса	ValidationPipe (forbidNonWhitelisted)	400	ошибка валидации
Email не зарегистрирован	AuthService.login	400	«Invalid email or password» (намеренно обобщённое)
Email зарегистрирован, пароль неверный	AuthService.login	400	то же обобщённое сообщение
Логин/пароль верные, но аккаунт не ACTIVE	AuthService.login	403	«This account is not active»
Сеть/CORS/бэкенд недоступен	Axios (нет response)	—	сообщение по умолчанию из thunk'a, напр. «Could not Sign in»
Необработанное исключение на сервере	AllExceptionsFilter	500	«Internal server error» (реальная ошибка — только в логах сервера)

Во всех случаях с 400/403/500 функция `getAxiosErrorMessage` читает JSON-тело ошибки, сформированное `AllExceptionsFilter`, и превращает его в простую строку под формой входа; ничего не «роняет» UI.

12. Механизм тихого обновления токена (refresh)

Не относится напрямую к самой форме входа, но напрямую использует токены, ею выданные (platform-admin/src/lib/axios.ts):

- Любой *авторизованный* запрос, вернувшийся с `401` (и ещё не являющийся повтором, и содержавший заголовок Authorization), запускает один общий вызов `POST /auth/refresh` с сохранённым refresh-токеном.
- `AuthService.refresh` заново проверяет подпись JWT, загружает сессию, проверяет, что она не отозвана и не истекла, сверяет HMAC предъявленного refresh-токена с сохранённым хешем через `timingSafeEqual` — сравнение за постоянное время, чтобы не утекала информация о том, сколько байт хеша совпало, — затем перевыпускает оба токена и обновляет ту же строку сессии.
- Несколько параллельных 401-ответов используют один и тот же ожидающий вызов обновления (`pendingRefresh`), а не запускают несколько запросов одновременно.
- Если само обновление токена не удастся, сессия удаляется из `localStorage`, и браузер жёстко перенаправляется на `/auth/login`.

13. Итоговые свойства безопасности

Нет enumeration через тайминг: запрос с несуществующим email всё равно «платит» полную стоимость bcrypt, сравнивая пароль с фиктивным хешем.

Нет enumeration через сообщение: неверный email и неверный пароль дают идентичную строку ошибки и одинаковый статус-код.

Refresh-токены хранятся хешированными (HMAC-SHA256), а не как открытый JWT в базе данных.

Сравнение за постоянное время (`timingSafeEqual`) при проверке предъявленного refresh-токена против сохранённого хеша.

Разные секреты подписи для access- и refresh-токенов.

Сессии отзываемы на сервере (`revokedAt`) независимо от срока жизни JWT, и каждый авторизованный запрос заново проверяет строку сессии, а не только подпись JWT.

Хеш пароля и секрет 2FA никогда не возвращаются в `/auth/me` (`omit` в запросе Prisma).

Строгая валидация входных данных (`whitelist` + `forbidNonWhitelisted`) отклоняет любое неожиданное поле в теле запроса ещё до бизнес-логики.

Обобщённые сообщения 500: любое неожиданное/внутреннее исключение никогда не пересылается клиенту дословно, а только логируется на сервере.